Plateforme DevSecOps de déploiement continu sur Kubernetes

Rapport technique — projet personnel

Ghassen KAHIA — élève ingénieur en Cloud Computing

Octobre 2026

Table des matières

1 Introduction générale **3**

2 Contexte et problématique **3**
2.1 Situation de départ (scénario) . . . 3
2.2 Risques associés . . . 3
2.3 Problématique . . . 4
2.4 Objectifs . . . 4

3 État de l'art **4**
3.1 DevOps . . . 4
3.2 DevSecOps . . . 4
3.3 GitOps . . . 5
3.4 Kubernetes et ses distributions légères . . . 5
3.5 Comparaison des outils de CI . . . 5
3.6 Comparaison des outils GitOps . . . 6
3.7 Autres comparaisons ayant guidé les choix . . . 6

4 Analyse des besoins et conception **7**
4.1 Besoins fonctionnels . . . 7
4.2 Besoins non fonctionnels . . . 7
4.3 Architecture générale . . . 7
4.4 Choix de conception structurants . . . 10
4.5 Planification . . . 10

5 Réalisation **10**
5.1 Phase 0 — Audit de l'environnement . . . 10
5.2 Phase 1 — Application de démonstration . . . 10
5.3 Phase 2 — Conteneurisation sécurisée . . . 11
5.4 Phase 3 — Infrastructure as Code . . . 11
5.5 Phase 4 — Kubernetes et Helm . . . 12
5.6 Phase 5 — Pipeline CI DevSecOps . . . 12
5.7 Phase 6 — GitOps avec Argo CD et Sealed Secrets . . . 12
5.8 Phase 7 — Supervision . . . 13
5.9 Phase 8 — Documentation et reproductibilité . . . 13
5.10 Synthèse : défense en profondeur . . . 13

6 Tests, résultats et indicateurs mesurés **14**
6.1 Qualité et tests automatisés . . . 14
6.2 Sécurité des images (Trivy, toutes sévérités) . . . 14
6.3 Efficacité des contrôles bloquants (contrôles positifs) . . . 15
6.4 Délais de déploiement et de retour arrière . . . 15
6.5 Disponibilité pendant les changements . . . 15
6.6 Correction des dérives (self-heal) . . . 16
6.7 Supervision : détection d'une panne contrôlée . . . 16
6.8 Autoscaling et quotas . . . 16
6.9 Consommation de ressources . . . 16

7 Difficultés rencontrées et solutions **17**
7.1 Erreurs 502 pendant une release défectueuse . . . 17
7.2 Erreurs 502 pendant une mise à jour réussie . . . 17
7.3 Le piège du rollback hors de l'outil de déploiement . . . 17
7.4 Faux positifs et exceptions de sécurité . . . 17
7.5 Erreurs de ma part et garde-fous ajoutés . . . 18
7.6 Incident : gel de la VM et cascade de redémarrages . . . 18
7.7 Contraintes de l'outillage . . . 18

8 Conclusion et perspectives **18**
8.1 Bilan . . . 18
8.2 Limites . . . 19
8.3 Perspectives . . . 19

9 Bibliographie **19**

10 Annexe — Correspondance avec le dépôt **21**

1 Introduction générale

Livrer vite et livrer sûr sont souvent présentés comme deux objectifs contradictoires. Les équipes qui déploient à la main multiplient les étapes répétitives, et donc les erreurs. Celles qui automatisent sans contrôles risquent de propager plus vite des vulnérabilités jusqu'en production.

Ce rapport présente un **projet personnel** réalisé pour mon portfolio, dans le cadre de ma recherche de stage de fin d'études en Cloud, DevOps ou DevSecOps. Il consiste à concevoir, construire, sécuriser et superviser une plateforme complète. Chaque modification poussée sur GitHub y est automatiquement testée, analysée, publiée sous forme d'image de conteneur, puis déployée sur un cluster Kubernetes par une approche GitOps, sous supervision et avec alertes.

Le projet a été mené sur une machine virtuelle Ubuntu personnelle (2 vCPU, 11 Gio de RAM), avec une contrainte de coût nul. Il ne s'agit pas d'un projet réalisé en entreprise : les « utilisateurs » sont simulés, et les incidents analysés sont ceux réellement survenus pendant la construction de la plateforme.

Convention du rapport. Les valeurs chiffrées proviennent toutes de mesures réelles, consignées avec leur méthode, leur date et leurs limites dans `docs/mesures.md`. Lorsqu'un élément relève d'un objectif, d'une hypothèse ou d'une perspective, il est explicitement présenté comme tel.

Le dépôt public est consultable à l'adresse : https://github.com/Ghassen-kahia/plateforme-devsecops.

2 Contexte et problématique

2.1 Situation de départ (scénario)

Le scénario retenu est celui d'une équipe informatique qui :

- déploie son application **manuellement**, par une suite de commandes exécutées par un opérateur ;
- n'a **aucun contrôle de sécurité automatique** : ni détection de secrets, ni analyse de code, ni analyse de vulnérabilités des images ;
- n'a pas de **stratégie de retour arrière** maîtrisée ;
- n'a pas de **supervision centralisée** : les pannes sont découvertes par les utilisateurs.

2.2 Risques associés

Risque	Conséquence
Erreur humaine pendant un déploiement	Indisponibilité, configuration incohérente entre environnements
Secret commité dans le dépôt	Compromission de comptes, fuite de données
Dépendance ou image vulnérable	Exploitation d'une faille connue (CVE) en production
Modification manuelle non tracée	Dérive entre ce qui est documenté et ce qui tourne
Absence d'alerte	Temps de détection long, incident découvert par les clients

2.3 Problématique

Comment automatiser le déploiement d'une application sur Kubernetes de façon **reproductible**, en intégrant des **contrôles de sécurité bloquants** à chaque étape, une **stratégie de rollback** vérifiée et une **supervision avec alertes**, sans coût d'infrastructure ?

2.4 Objectifs

1. Chaque `git push` déclenche une chaîne automatisée : qualité, tests, secrets, SAST, construction, analyse des images et de l'infrastructure, publication, déploiement.
2. Les contrôles **bloquent** effectivement le pipeline lorsqu'une politique est violée, ce qui doit être démontré.
3. Le déploiement repose sur GitOps, avec correction automatique des dérives et retour arrière.
4. La plateforme est supervisée, et une panne est détectée et notifiée.
5. Les résultats sont **mesurés**, et non supposés.

3 État de l'art

3.1 DevOps

Le mouvement DevOps vise à réduire la distance entre développement et exploitation grâce à l'automatisation, à la mesure et au partage de responsabilités. Les travaux du programme DORA, synthétisés dans *Accelerate* (Forsgren, Humble, Kim, 2018), relient la performance des équipes à quatre indicateurs : fréquence de déploiement, délai de mise en production, taux d'échec des changements et temps de restauration. Ce projet en mesure des équivalents à son échelle : délai entre le push et la mise en service, temps de rollback et temps de détection.

3.2 DevSecOps

Le DevSecOps intègre la sécurité **dans** la chaîne, au lieu de l'ajouter en fin de cycle (« shift left »). Chaque type de contrôle cible une famille de risques :

Contrôle	Question posée	Outil retenu
Détection de secrets	Un mot de passe ou un jeton a-t-il été commité ?	Gitleaks
SAST (analyse statique)	Le code contient-il des motifs dangereux (injection, `shell=True`…) ?	Semgrep
SCA (dépendances)	Une bibliothèque utilisée a-t-elle une CVE connue ?	Trivy (`fs`)
Analyse d'image	Les paquets de l'image ont-ils des CVE ?	Trivy (`image`)
Analyse IaC	Terraform, Kubernetes ou Dockerfile ont-ils des erreurs de configuration ?	Trivy (`config`)
Chaîne d'approvisionnement	Les outils et actions utilisés sont-ils intègres et figés ?	SHA, sommes de contrôle

Le cadre SLSA (*Supply-chain Levels for Software Artifacts*) et le guide NIST SP 800-190 sur la

sécurité des conteneurs ont servi de références pour la chaîne d'approvisionnement et le durcissement des images.

3.3 GitOps

GitOps, formalisé par Weaveworks en 2017 puis par le groupe de travail OpenGitOps (CNCF), repose sur quatre principes : un état **déclaratif**, **versionné** et immuable dans Git, **tiré** (pull) automatiquement par un agent, puis **réconcilié en continu**. La différence avec un pipeline qui exécute `kubectl apply` (modèle push) est essentielle pour la sécurité : la CI n'a plus besoin d'identifiants sur le cluster.

3.4 Kubernetes et ses distributions légères

Kubernetes orchestre des conteneurs à partir d'un état désiré. Plusieurs distributions conviennent à un environnement local :

Critère	k3s	minikube	kind	kubeadm
Nature	Distribution certifiée, binaire unique	Cluster dans une VM ou un conteneur	Nœuds dans des conteneurs Docker	Outil d'installation « vanilla »
Empreinte	Faible	Moyenne	Faible	Élevée
Composants inclus	Traefik, CoreDNS, metrics-server, stockage local, NetworkPolicy	Modules complémentaires	Minimal	Aucun
Usage typique	Edge, IoT, nœuds légers, production possible	Développement	Tests CI éphémères	Clusters « réels »
Proximité d'un vrai nœud	Forte (service systemd)	Moyenne	Faible	Forte

Choix : k3s. Faible consommation (critique avec 2 vCPU), composants utiles inclus et installation comparable à un serveur réel, automatisable par Ansible.

3.5 Comparaison des outils de CI

Critère	Jenkins	GitHub Actions	GitLab CI
Hébergement	Auto-hébergé (serveur à maintenir)	SaaS (runners GitHub) ou auto-hébergé	SaaS ou auto-hébergé
Configuration	Jenkinsfile (Groovy) + plugins	YAML dans `.github/workflows`	YAML `.gitlab-ci.yml`
Écosystème	Très large catalogue de plugins, qualité variable	Place de marché d'actions	Intégré à la plateforme GitLab
Coût pour un dépôt public	Coût du serveur	Gratuit (runners hébergés)	Quotas de minutes

Critère	Jenkins	GitHub Actions	GitLab CI
Sécurité de la chaîne	Plugins à maintenir, surface d'attaque du serveur	Jetons éphémères, permissions par job, épinglage par SHA	Jetons de job, environnements protégés
Intégration au registre	Plugins	GHCR natif (`GITHUB_TOKEN`)	Registre GitLab natif

Choix : GitHub Actions. Le code est hébergé sur GitHub, l'intégration avec GHCR passe par un jeton éphémère (aucun secret longue durée), il n'y a aucun serveur à opérer et le service est gratuit pour un dépôt public. Jenkins aurait ajouté un serveur à maintenir sur une VM déjà contrainte.

3.6 Comparaison des outils GitOps

Critère	Argo CD	Flux
Statut CNCF	Projet diplômé	Projet diplômé
Interface	Interface web riche (graphe des ressources, historique)	Pas d'interface officielle (CLI, tableaux de bord tiers)
Architecture	Serveur + contrôleur + repo-server	Ensemble de contrôleurs modulaires (GitOps Toolkit)
Multi-équipes	AppProject (sources, destinations, types de ressources)	Isolation par namespaces et comptes de service
Secrets	Via outils externes (Sealed Secrets, plugins)	Intégration native avec SOPS
Mise à jour automatique des images	Argo CD Image Updater (projet séparé)	Contrôleurs d'images intégrés

Choix : Argo CD, pour sa visibilité pédagogique (utile en démonstration) et les AppProjects, qui permettent d'appliquer le moindre privilège à l'outil de déploiement lui-même.

3.7 Autres comparaisons ayant guidé les choix

Besoin	Retenu	Alternatives	Raison principale
Scan d'images et d'IaC	Trivy	Grype + Syft, Snyk, Docker Scout	Un seul outil pour images, dépendances, IaC et secrets ; gratuit, sans serveur
SAST	Semgrep	SonarQube, CodeQL	CLI sans serveur ; SonarQube demande un serveur et une base (environ 2 Go RAM)
Secrets dans Git	Sealed Secrets	SOPS, External Secrets + Vault	Aucun service externe ; chiffrement asymétrique propre au cluster

Besoin	Retenu	Alternatives	Raison principale
Supervision	kube-prometheus-stack	Prometheus seul, VictoriaMetrics, offres SaaS	Standard du marché, CRD (ServiceMonitor, PrometheusRule)
Base de l'image Python	Debian slim	Alpine, distroless	Roues binaires glibc, Python 3.12 conservé, débogage possible

4 Analyse des besoins et conception

4.1 Besoins fonctionnels

— **BF1** — Application web simple (API + interface) persistant des données dans PostgreSQL.
— **BF2** — Endpoints d'exploitation : vivacité (`/health`), disponibilité (`/ready`), métriques (`/metrics`).
— **BF3** — Deux environnements : dev (déploiement automatique) et prod (promotion manuelle).
— **BF4** — Retour arrière vers une version antérieure.

4.2 Besoins non fonctionnels

Exigence	Traduction technique	Vérification
Sécurité	Contrôles bloquants, conteneurs non-root, réseau en deny-all, secrets chiffrés	PR bloquées, matrice de flux réseau, PSA
Reproductibilité	IaC, versions figées, procédures idempotentes	Clone neuf, `--check`, `plan` sans changement
Disponibilité	Probes, RollingUpdate sans indisponibilité, HPA	Sonde continue pendant les déploiements
Observabilité	Métriques RED, alertes, tableau de bord	Panne contrôlée
Coût	0 €	Aucun service payant utilisé
Contraintes matérielles	2 vCPU / 11 Gio	Requests/limits, profils réduits, mesures de consommation

4.3 Architecture générale

La chaîne se découpe en trois zones de confiance :

1. **GitHub** (code, CI, registre GHCR) : la CI teste, analyse, construit et publie. Elle écrit l'état désiré dans Git, mais **n'a aucun accès au cluster**.
2. **Git** comme source de vérité : `k8s/environments/<env>/values.yaml` décrit la version et la configuration de chaque environnement.
3. **Le cluster k3s** : Argo CD lit Git et applique ; Sealed Secrets déchiffre les secrets ; Prometheus surveille.

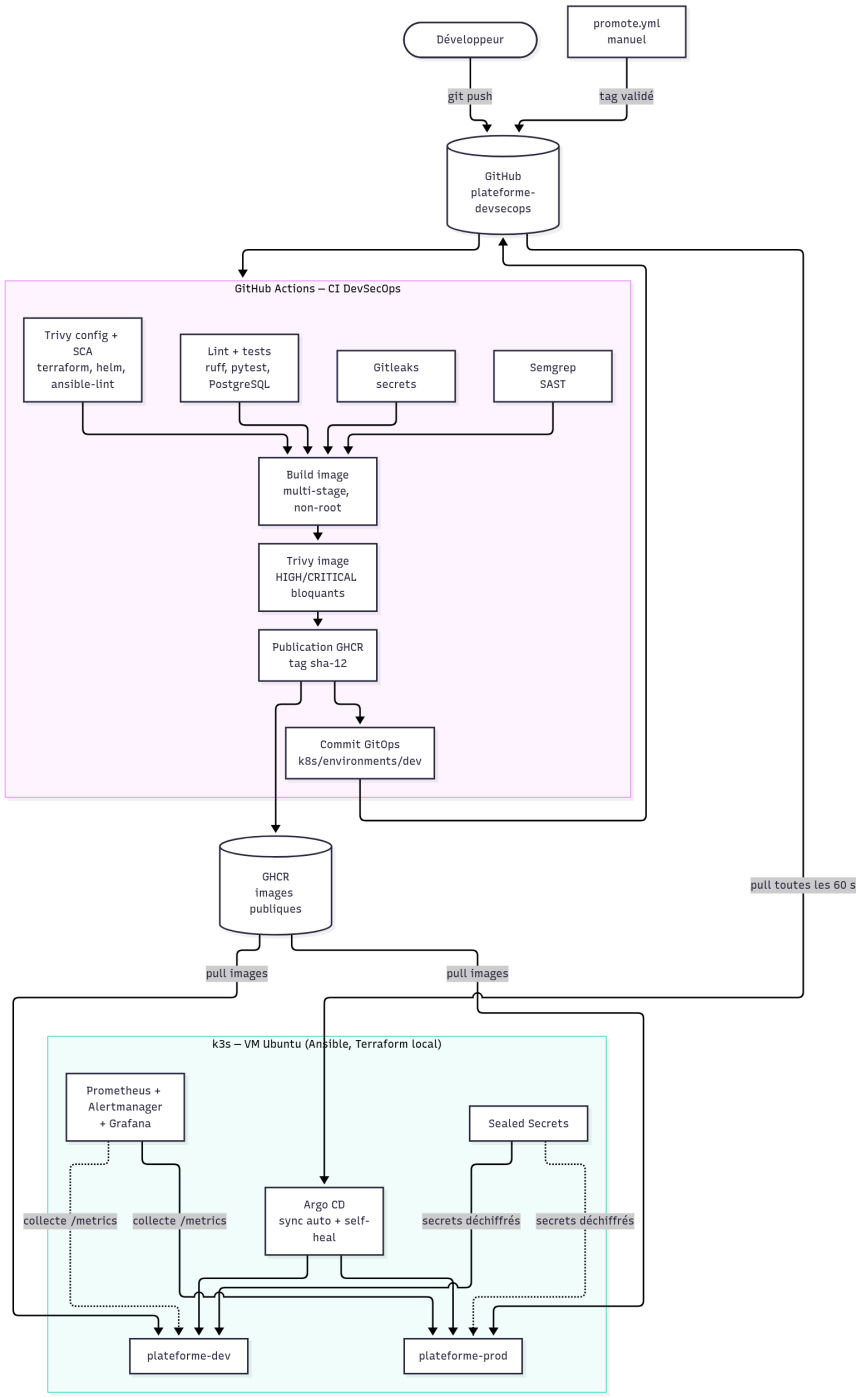


FIGURE 1 – Chaîne DevSecOps : du commit au cluster

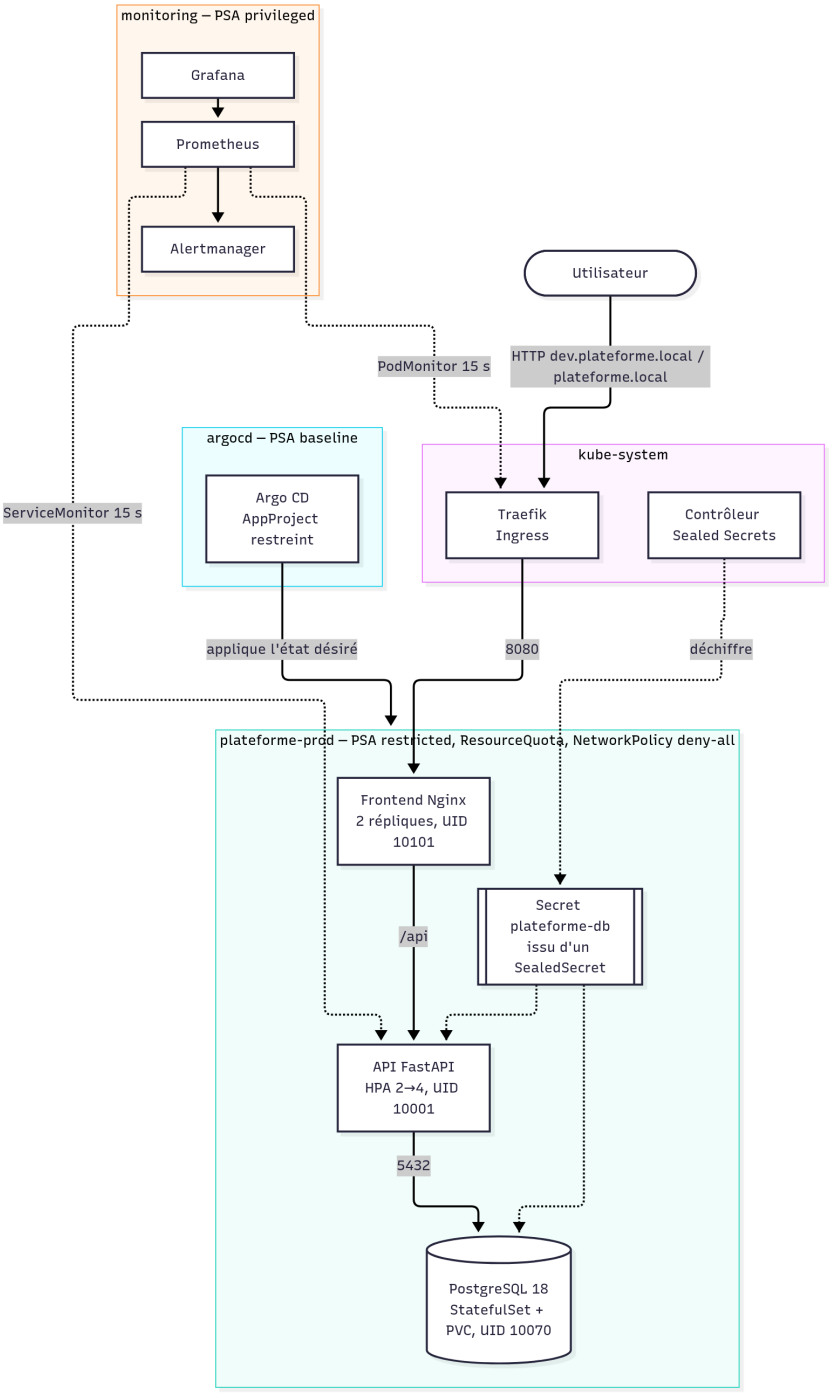


FIGURE 2 – Architecture d'exécution dans k3s

4.4 Choix de conception structurants

— **Liveness et readiness séparées** : `/health` ne dépend pas de la base, `/ready` la teste. Une panne de base retire les pods du trafic sans les redémarrer en boucle.
— **Pattern repository** dans l'API : tests unitaires en mémoire et tests d'intégration sur un vrai PostgreSQL.
— **Image immuable promue** : la même image (`sha-<12 caractères du commit>`), construite et scannée une seule fois, est déployée en dev puis promue en prod.
— **Supervision livrée avec l'application** : ServiceMonitor et règles d'alerte font partie du chart.
— **Défense en profondeur** : aucune mesure n'est supposée suffisante seule (voir la section 5.8).

4.5 Planification

Le projet a été découpé en dix phases, chacune validée et commitée avant la suivante : audit, application, conteneurisation, IaC, Kubernetes, CI, GitOps, supervision, documentation, rapport et présentation.

5 Réalisation

5.1 Phase 0 — Audit de l'environnement

Avant toute installation, la VM a été auditée : Ubuntu 24.04.5, **2 vCPU**, 11 Gio de RAM, 42 Go de disque libres, cgroups v2 et UFW installé mais inactif. Deux risques ont été identifiés dès cette étape et se sont confirmés par la suite : **la contrainte CPU**, et le fait que **Docker contourne UFW** pour les ports publiés (d'où la publication sur `127.0.0.1` dans Compose).

Les outils sont installés « juste à temps » par `scripts/install.sh`, en une cible par phase, depuis les dépôts APT officiels ou depuis des binaires de release dont la **somme SHA-256 est vérifiée** :

```
install_github_release() {
  local repo=$1 tag=$2 archive=$3 sums=$4 bin=$5 tmp
  tmp=$(mktemp -d)
  curl -fsSL -o "$tmp/$archive" "https://github.com/$repo/releases/download/$tag/$archive"
  curl -fsSL -o "$tmp/sums.txt" "https://github.com/$repo/releases/download/$tag/$sums"
  (cd "$tmp" && grep " ${archive}\$" sums.txt | sha256sum -c -) || fail "somme SHA-256 invalide"
  ...
}
```

5.2 Phase 1 — Application de démonstration

L'API FastAPI gère des tâches stockées dans PostgreSQL (psycopg 3 avec un pool de connexions). Le frontend statique est servi par Nginx, qui relaie `/api` vers l'API : le navigateur ne voit qu'une seule origine, ce qui évite toute configuration CORS.

Métriques. Le middleware expose la méthode RED et utilise le **modèle** de route comme label, afin de maîtriser la cardinalité :

```
route = request.scope.get("route")
route_label = getattr(route, "path", "<non_routé>")    # /api/tasks/{task_id}, pas /api/tasks/42
HTTP_REQUESTS.labels(request.method, route_label, str(status)).inc()
HTTP_LATENCY.labels(request.method, route_label).observe(time.perf_counter() - start)
```

Sécurité applicative : requêtes SQL paramétrées (testées avec une charge d'injection), `SecretStr` pour le mot de passe, message 503 générique (le détail reste dans les logs), en-têtes de sécurité Nginx (CSP, `X-Frame-Options`, `nosniff`) et `textContent` côté JavaScript contre le XSS.

Tests : 16 cas unitaires (dépôt en mémoire) et 4 tests d'intégration contre un vrai PostgreSQL, avec 98 % de couverture lors de la première mesure. Une panne de base simulée en local a confirmé le comportement attendu : `/health` à 200, `/ready` et l'API à 503, reprise en environ 1 s.

`[CAPTURE : interface web de l'application avec quelques tâches]`

5.3 Phase 2 — Conteneurisation sécurisée

La première image, volontairement naïve (`FROM python:3.12.15`, exécution en root), a servi de point de référence. L'image durcie applique :

```
FROM ${PYTHON_IMAGE} AS build                    # python:3.12.15-slim-trixie
RUN python -m venv /opt/venv
RUN /opt/venv/bin/pip install --no-compile -r /tmp/requirements.txt \
 && /opt/venv/bin/pip uninstall -y pip
FROM ${PYTHON_IMAGE} AS runtime
RUN groupadd --gid 10001 app && useradd --uid 10001 --gid app --no-create-home \
      --shell /usr/sbin/nologin app && python -m pip uninstall -y --root-user-action=ignore pip
COPY --from=build /opt/venv /opt/venv
COPY --chown=root:root backend/api ./api        # code non modifiable par le processus
USER 10001:10001
```

Le `.dockerignore` fonctionne en **liste blanche** (`*` puis exceptions), et Compose ajoute un système de fichiers en lecture seule, `cap_drop: ALL` et `no-new-privileges`.

La politique Trivy (`trivy.yaml`) bloque les vulnérabilités **HIGH et CRITICAL corrigeables**. Celles qui n'ont pas de correctif sont tracées sans être bloquantes : les bloquer rendrait le pipeline rouge en permanence, sans action possible, et pousserait l'équipe à contourner le contrôle.

5.4 Phase 3 — Infrastructure as Code

- **Terraform AWS** (non appliqué) : VPC, sous-réseau public, groupe de sécurité dont l'administration (SSH, 6443) est restreinte à `admin_cidr` (une validation refuse `0.0.0.0/0`), EC2 avec **IMDSv2 obligatoire** et disque chiffré, Flow Logs dont le rôle IAM est limité à son groupe de logs. Validation : `fmt`, `validate`, Trivy. Une seule exception justifiée subsiste (AWS-0104, sortie 80/443).
- **Terraform local** (appliqué en Phase 4) : namespaces avec Pod Security Admission (`restricted` pour les applications), ResourceQuota et LimitRange.
- **Ansible** : rôles `common` (paquets, sysctl inspirés de CIS), `ssh_hardening` (si un serveur SSH est présent), `firewall` (UFW, réseaux k3s) et `k3s`. Le script d'installation de k3s est **figé au tag de version et vérifié par SHA-256**, au lieu du classique `curl | sh` :

```
- name: Télécharger le script d'installation k3s (somme SHA-256 vérifiée)
  ansible.builtin.get_url:
    url: "https://raw.githubusercontent.com/k3s-io/k3s/{{ k3s_version | urlencode }}/install.sh"
    checksum: "sha256:{{ k3s_install_script_sha256 }}"
```

k3s est configuré avec `secrets-encryption`, `protect-kernel-defaults` et un journal d'audit de l'API. La validation repose sur ansible-lint (profil *production*) et une exécution à blanc (`--check`).

5.5 Phase 4 — Kubernetes et Helm

Un chart unique (`k8s/helm/plateforme`) décrit l'API, le frontend, PostgreSQL (StatefulSet), l'Ingress Traefik, le HPA et les NetworkPolicy. Les valeurs propres à chaque environnement se trouvent dans `k8s/environments/<env>/values.yaml`.

Contexte de sécurité commun, conforme au niveau PSA *restricted* : non-root (UID > 10000), `allowPrivilegeEscalation: false`, `drop: [ALL]`, seccomp `RuntimeDefault`, rootfs en lecture seule (volumes `emptyDir` pour les seuls chemins inscriptibles) et aucun jeton d'API monté.

Réseau : une politique *deny-all*, puis cinq flux explicites (DNS, Traefik → frontend, frontend → API, monitoring → API, API → PostgreSQL). Leur application effective par le contrôleur de k3s a été **prouvée par un contrôle positif** : le même flux est autorisé sans policy et bloqué avec.

`[CAPTURE : kubectl get pods,svc,ingress,netpol -n plateforme-prod]`

5.6 Phase 5 — Pipeline CI DevSecOps

`.github/workflows/ci.yml` enchaîne quatre jobs parallèles (lint/tests avec PostgreSQL de service, Gitleaks, Semgrep, IaC/SCA), puis la construction, le scan et la publication de chaque image, et enfin le commit GitOps. Principes appliqués :

```yaml
permissions: {}                          # aucune permission par défaut
jobs:
  build-scan-push:
    permissions:
      contents: read
      packages: write                    # uniquement pour publier dans GHCR
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
```

- actions **épinglées par SHA** (un tag peut être déplacé par un attaquant) ;
- outils installés à version fixe avec somme vérifiée, en choisissant des versions publiées depuis **au moins deux semaines**, sauf correctif de sécurité (voir Argo CD en Phase 6) ;
- `GITHUB_TOKEN` éphémère pour GHCR et pour le commit GitOps : **aucun secret longue durée** ;
- SBOM CycloneDX et rapport Trivy complet conservés en artefacts.

`[CAPTURE : pipeline GitHub Actions réussi (vue des 7 jobs)]`

`[CAPTURE : PR #1 bloquée par Gitleaks - valeur REDACTED dans les logs]`

`[CAPTURE : PR #2 bloquée par Trivy - CVE-2020-14343]`

5.7 Phase 6 — GitOps avec Argo CD et Sealed Secrets

- **Application racine** (*app of apps*) qui gère les applications `plateforme-dev` et `plateforme-prod`, avec synchronisation automatique, `prune` et `selfHeal`.
- **AppProject restreint** : un seul dépôt source, deux namespaces de destination, aucune ressource de niveau cluster et une liste blanche de types d'objets.
- **Sealed Secrets** : le mot de passe PostgreSQL est versionné **chiffré**, avec une portée `strict` (liée au nom et au namespace). La migration a chiffré la valeur existante par un pipe, sans jamais l'afficher, et son empreinte SHA-256 était identique avant et après.
- **Promotion** : `promote.yml`, déclenché manuellement, recopie en prod le tag validé en dev.

Argo CD a été installé en **version 3.5.4** et non en 3.5.3, alors que cette dernière respectait la règle des deux semaines de recul : la 3.5.3 est affectée par quatre vulnérabilités critiques publiées le 7 octobre 2026. Une règle de prudence ne doit pas primer sur un correctif de sécurité.

`[CAPTURE : interface Argo CD - applications Synced/Healthy et arbre des ressources]`

5.8 Phase 7 — Supervision

kube-prometheus-stack est installé avec un profil réduit, et les cibles du plan de contrôle, inexistantes dans k3s, sont désactivées. L'application livre son ServiceMonitor et ses quatre règles d'alerte. Point clé : l'alerte sur les erreurs 5xx est mesurée **au niveau de Traefik**. Quand l'API est retirée du Service, ses propres métriques ne voient plus les requêtes en erreur.

```
- alert: PlateformeTauxErreurs5xxEleve
  expr: |
    sum(rate(traefik_service_requests_total{service="...-frontend-http@kubernetes", code=~"5.."}
      / sum(rate(traefik_service_requests_total{service="...-frontend-http@kubernetes"}[2m])) >
  for: 1m
  labels:
    severity: critical
    namespace: plateforme-prod      # rétabli : les agrégations suppriment ce label
```

Le tableau de bord Grafana (12 panneaux : RED, état de la base, répliques, version, ressources) est provisionné depuis Git. Le mot de passe administrateur par défaut du chart, connu publiquement, est remplacé par un mot de passe aléatoire, et le refus de l'ancien a été vérifié (HTTP 401).

`[CAPTURE : tableau de bord Grafana « Plateforme - Application »]`

`[CAPTURE : Alertmanager pendant la panne contrôlée - 3 alertes critiques]`

5.9 Phase 8 — Documentation et reproductibilité

Cette phase a produit le README professionnel, les diagrammes Mermaid exportés en PNG et `scripts/demo.sh`, qui propose six scénarios dont une panne contrôlée protégée par une confirmation explicite. La reproductibilité a été vérifiée sur un **clone neuf depuis GitHub** (douze contrôles rejoués) et par la réexécution idempotente des procédures : Ansible `--check` avec 0 changement, Terraform `plan` sans changement.

5.10 Synthèse : défense en profondeur

Couche	Mesures appliquées
Code	SAST, tests (injection SQL), SecretStr, messages d'erreur génériques
Dépendances	Versions figées, SCA bloquante, recul de deux semaines (sauf correctif de sécurité)
Image	Base slim, non-root, sans pip, scan bloquant, SBOM
CI	Permissions minimales, SHA épinglés, jetons éphémères, aucun accès au cluster
Cluster	PSA restricted, securityContext, deny-all réseau, quotas, aucun jeton d'API

Couche	Mesures appliquées
Secrets	Sealed Secrets (portée strict), chiffrement au repos par k3s
Déploiement	GitOps, AppProject restreint, self-heal
Hôte	UFW, sysctl CIS, k3s durci (audit, secrets chiffrés)
Détection	Prometheus, 4 alertes par environnement, Alertmanager

6 Tests, résultats et indicateurs mesurés

Environnement de toutes les mesures : VM VMware, Ubuntu 24.04.5, 2 vCPU (Intel i5-11400H), 11 Gio de RAM, k3s v1.36.5, mesures réalisées le 14 octobre 2026. Les mesures sont ponctuelles (généralement un seul essai par scénario) : elles donnent un **ordre de grandeur**, pas une moyenne statistique.

6.1 Qualité et tests automatisés

Mesure	Résultat
Tests unitaires	16 réussis (dont 4 cas paramétrés)
Tests d'intégration (PostgreSQL réel)	4 réussis
Couverture de code (première mesure)	98 % (seuil bloquant en CI : 90 %)
Lint Python (ruff, règles de sécurité incluses)	0 erreur
ansible-lint (profil production)	0 échec

6.2 Sécurité des images (Trivy, toutes sévérités)

Image	CRITICAL	HIGH	Total	Taille	Utilisateur
API naïve (`python:3.12.15`)	16	313	3 695	1,69 GB	root
API durcie (slim, multi-stage)	**0**	44*	**166**	**221 MB**	10001
Frontend naïf (`nginx:1.30.5`)	1	67	401	240 MB	root
Frontend durci (unprivileged, Alpine)	**0**	**0**	**1**	**90,1 MB**	101 puis 10101

* Les 44 HIGH restantes sont toutes **sans correctif publié** par Debian (paquets *Essential* : util-linux, ncurses, libsystemd, perl-base). Elles sont tracées, et non bloquantes selon la politique.

Les dépendances applicatives (FastAPI, psycopg…) ne présentaient **aucune** vulnérabilité : tout le risque venait de l'image de base.

6.3 Efficacité des contrôles bloquants (contrôles positifs)

Un scanner silencieux ne prouve rien : chaque contrôle a été confronté à un cas qu'il **doit** bloquer.

Contrôle	Cas de test	Résultat
Gitleaks (CI)	PR #1 : clé factice aléatoire	**Bloqué**, valeur masquée dans les logs
Gitleaks	Secret supprimé dans un commit suivant	**Toujours bloqué** (le secret reste dans l'historique)
Trivy fs (CI)	PR #2 : `pyyaml==5.3.1`	**Bloqué** (CVE-2020-14343, CRITICAL)
Trivy image	Image `python:3.12.15`	**Bloqué** (12 HIGH corrigeables)
Semgrep	Fichier volontairement vulnérable	4 lignes sur 5 détectées ; `yaml.load` non détecté (limite)
PSA	Pod `nginx` standard dans un namespace restricted	**Refusé** (4 violations)
NetworkPolicy	Autre namespace → PostgreSQL	Autorisé sans policy, **bloqué** avec
Sealed Secrets	Texte chiffré rejoué dans un autre namespace	**Refusé**

6.4 Délais de déploiement et de retour arrière

Scénario	Résultat
Déploiement manuel (build, import, secret, `helm --wait`)	84 s, **sans aucun contrôle**
Pipeline CI complet (7 jobs)	125 à 145 s selon les runs
`git push` → nouvelle version servie en dev	**189 s**, sans intervention
Configuration modifiée dans Git → active en dev	49 s
Promotion prod (`promote.yml`) → nouvelle version servie	84 s
Rollback GitOps (`git revert`) → ancienne version servie	**89 s**
Rollback Helm (Phase 4, avant GitOps)	0,6 à 2,5 s

Interprétation. Le déploiement automatisé n'est pas plus rapide que le déploiement manuel, et ce n'est pas l'objectif. Pour un délai du même ordre (189 s contre 84 s), il exécute 20 tests, cinq familles d'analyses de sécurité, un SBOM et une publication tracée, **sans intervention humaine** et avec un historique complet dans Git. Le délai GitOps dépend aussi de l'intervalle de comparaison d'Argo CD (60 s), qu'un webhook réduirait si GitHub pouvait joindre la VM.

6.5 Disponibilité pendant les changements

Une sonde HTTP interroge le service toutes les 0,3 s via l'Ingress pendant chaque opération.

Opération	Requêtes	Erreurs
Release défectueuse + rollback, **avant** correction du chart	99	10 (502)
Idem **après** correction (2 essais)	365	**0**
Reprise par Argo CD, **avant** ajout du `preStop`	163	3
Changement de configuration GitOps (avec `preStop`)	939	**0**
Rollback d'image GitOps	1 460	**0**
Promotion vers prod	938	**0**

6.6 Correction des dérives (self-heal)

Modification manuelle	Corrigée par Argo CD en
`kubectl scale` du frontend (1 → 3)	1,7 s
Suppression de la NetworkPolicy de PostgreSQL	2,0 s
Modification d'une ConfigMap	6,1 s

6.7 Supervision : détection d'une panne contrôlée

Panne déclenchée par un commit Git (`database.replicas: 0`) avec un trafic de 3 requêtes par seconde.

Jalon (référence : première erreur vue par l'utilisateur)	Délai
Alertes `pending`	+34 s
`BaseDeDonneesIndisponible` reçue par Alertmanager	**+63 s**
Les trois alertes critiques reçues par Alertmanager	**+94 s**
`git revert` → service rétabli	166 s
`git revert` → plus aucune alerte active	359 s

6.8 Autoscaling et quotas

Mesure	Résultat
HPA sous charge (24 clients, 180 s)	2 → 4 répliques en environ 34 s
Retour à 2 répliques	121 s (fenêtre de stabilisation de 120 s)
ResourceQuota : 20 répliques demandées en dev	2 créées (limite `limits.cpu` atteinte)

6.9 Consommation de ressources

État	Mesure
VM au démarrage (bureau GNOME seul)	2,2 Gio de RAM
k3s + 2 environnements	environ 3,8 Gio sur le nœud, 9 % CPU

État	Mesure
+ Argo CD et Sealed Secrets	environ 4,8 Gio, 12 % CPU
+ supervision (environ 1 Go pour 6 pods)	environ 5,6 Gio ; **environ 67 à 73 % des 2 vCPU au repos**

La plateforme complète fonctionne sur 2 vCPU, mais avec une marge faible, ce qui s'est traduit par un incident réel (section 7). La recommandation formulée dès l'audit initial, passer à 4 vCPU, est confirmée.

7 Difficultés rencontrées et solutions

Les difficultés ci-dessous sont celles réellement rencontrées. Leur analyse détaillée figure dans `docs/apprentissage.md`.

7.1 Erreurs 502 pendant une release défectueuse

- **Symptôme** : 10 % d'erreurs pendant une release dont l'image était introuvable, alors que l'ancien pod restait en service.
- **Diagnostic** : la corrélation des journaux (Nginx, événements Kubernetes) montrait que **PostgreSQL redémarrait**. Le label `app.kubernetes.io/version`, qui suit la version de l'application, figurait dans le modèle de pod du StatefulSet.
- **Correction** : un modèle de pod PostgreSQL sans label de version, puis 0 erreur mesurée ensuite.
- **Piège méthodologique** : un second essai a d'abord montré 9 erreurs. Trois hypothèses testées isolément ont été réfutées, puis une bissection a montré que la mesure avait démarré trop tôt, avant la stabilisation. Il faut mesurer à partir d'un état stable.

7.2 Erreurs 502 pendant une mise à jour réussie

- **Cause** : Kubernetes envoie SIGTERM au conteneur **en même temps** qu'il le retire des endpoints. Pendant le temps de propagation, des requêtes arrivent encore sur un processus qui s'arrête.
- **Correction** : `preStop` (action native `sleep`, 5 s) et `terminationGracePeriodSeconds`. Résultat : 0 erreur sur 3 337 requêtes, sur trois déploiements.
- **Enseignement** : les tests de la Phase 4 n'arrêtaient jamais l'ancien pod. Un test ne couvre que ce qu'il exerce réellement.

7.3 Le piège du rollback hors de l'outil de déploiement

`kubectl rollout undo` suivi de `helm rollback` a produit un **état incohérent** (pod en WARNING, ConfigMap en DEBUG). Cette démonstration a directement motivé le passage à GitOps : une seule source de vérité, et les modifications manuelles sont annulées.

7.4 Faux positifs et exceptions de sécurité

- Un texte chiffré par Sealed Secrets était signalé comme « clé d'API » par Gitleaks. Une première exception par chemin était **trop large** : elle masquait aussi un mot de passe en clair dans le même fichier, ce qu'un contrôle positif a révélé. L'exception finale porte sur le format exact du texte chiffré.

— Le secret factice de la démonstration aurait fait échouer le scan hebdomadaire `--all`. L'exception est limitée au SHA exact du commit.
— Trivy 0.76 avait renommé ses identifiants de règles. Un scan sans exceptions a permis de retirer deux exceptions qui ne correspondaient à aucun constat réel.

7.5 Erreurs de ma part et garde-fous ajoutés

— **Un commit a été poussé malgré deux validations en échec**, parce que les étapes étaient chaînées avec `;` au lieu de `&&`. La CI l'a bloqué, et aucune image n'a été publiée. Depuis, les validations sont strictement chaînées et `gitleaks --staged` est lancé avant chaque commit.
— Une clé YAML mal indentée (le PodMonitor Traefik placé sous `grafana:`) a été **ignorée sans erreur**. Une vérification de structure a été ajoutée.

7.6 Incident : gel de la VM et cascade de redémarrages

— **Faits** : aucune ligne de journal système pendant 25 minutes, un trou équivalent dans Prometheus, puis au réveil `Node became not ready`, des évictions et une charge de 52 sur 2 vCPU.
— **Cause** : suspension de la VM par l'hôte. Au réveil, toutes les probes ont expiré en même temps, provoquant une vague de redémarrages qui a saturé le CPU.
— **Résolution** : rétablissement automatique en environ 8 minutes.
— **Enseignements** : une supervision interne ne peut pas signaler que sa propre machine est figée. C'est le rôle d'un *dead man's switch* externe (alerte `Watchdog`). La marge CPU était insuffisante, et sa mesure au repos (environ 70 %) a été documentée.

7.7 Contraintes de l'outillage

Parmi d'autres exemples : collections Ansible implicites (fournies par le paquet Ubuntu, absentes en CI), clé `applicationSet.enabled` inexistante dans le chart Argo CD, SIGPIPE avec `set -o pipefail`, port-forwards interrompus pendant les mesures longues (remplacés par le proxy de l'API Kubernetes) et erreur réseau transitoire d'un runner GitHub. Dans ce dernier cas, il ne fallait **pas** relancer l'ancien pipeline, sous peine de faire revenir dev à une image plus ancienne.

8 Conclusion et perspectives

8.1 Bilan

Les objectifs fixés sont atteints et **démontrés** :

— une chaîne automatisée du `git push` jusqu'au cluster, en environ 3 minutes, sans intervention ;
— des contrôles de sécurité bloquants, prouvés par des cas qui doivent échouer (secret factice, CVE critique, image vulnérable, pod root, flux réseau interdit) ;
— un déploiement GitOps avec correction des dérives en quelques secondes et un rollback par `git revert` ;
— une supervision qui détecte une panne de base de données en une à deux minutes ;
— des mises à jour sans erreur pour l'utilisateur, après correction de deux défauts mesurés.

Au-delà des outils, le projet m'a surtout appris une démarche : **mesurer plutôt que supposer**, confronter chaque contrôle à un cas positif, et documenter honnêtement les limites, y compris mes propres erreurs.

8.2 Limites

— Un seul nœud : pas de haute disponibilité réelle.
— 2 vCPU : marge insuffisante pour absorber une reprise après incident.
— Pas de TLS sur l'Ingress local, ni de canal de notification externe pour les alertes.
— État GitOps dans le dépôt applicatif : impossible d'exiger les checks sur `main`.
— Module AWS validé statiquement uniquement (non appliqué, par choix de coût).
— Mesures réalisées en un seul essai.

8.3 Perspectives

Axe	Évolution envisagée
Cloud	Appliquer le module Terraform AWS, puis migrer vers **EKS** avec Karpenter (autoscaling des nœuds)
Secrets	**Vault** ou AWS Secrets Manager avec External Secrets Operator (rotation centralisée)
Réseau	**Service mesh** (Istio, Linkerd) pour le mTLS entre services ; cert-manager pour le TLS
Chaîne d'approvisionnement	Signature des images (cosign), politique d'admission (Kyverno) n'acceptant que les images signées, provenance SLSA
GitOps	Dépôt GitOps séparé et `main` protégé ; Argo CD Image Updater ; webhook
Supervision	SLO et alertes par taux de consommation du budget d'erreur ; *dead man's switch* externe ; logs centralisés (Loki)
Images	Distroless ou Chainguard pour supprimer les vulnérabilités sans correctif

9 Bibliographie

Sources consultées pour la conception. Les documentations officielles ont été consultées en octobre 2026.

1. Forsgren N., Humble J., Kim G. *Accelerate : The Science of Lean Software and DevOps*. IT Revolution, 2018.
2. Beyer B., Jones C., Petoff J., Murphy N. R. (dir.). *Site Reliability Engineering*. O'Reilly, 2016 — chapitre « Monitoring Distributed Systems ». https://sre.google/sre-book/monitoring-distributed-systems/
3. Souppaya M., Morello J., Scarfone K. *Application Container Security Guide*. NIST SP 800-190, 2017. https://doi.org/10.6028/NIST.SP.800-190
4. OpenGitOps (CNCF). *GitOps Principles v1.0.0*. https://opengitops.dev/
5. SLSA. *Supply-chain Levels for Software Artifacts*. https://slsa.dev/
6. Center for Internet Security. *CIS Benchmarks* (Ubuntu Linux, Kubernetes). https://www.cisecurity.org/cis-benchmarks
7. k3s. *CIS Hardening Guide*. https://docs.k3s.io/security/hardening-guide

8. Kubernetes. *Pod Security Standards* et *Network Policies*. https://kubernetes.io/docs/concepts/security/pod-security-standards/, https://kubernetes.io/docs/concepts/services-networking/network-policies/
9. Kubernetes. *Configure Liveness, Readiness and Startup Probes*. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
10. GitHub. *Security hardening for GitHub Actions*. https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions
11. Argo CD. *Documentation* (Applications, AppProjects, sync policies). https://argo-cd.readthedocs.io/
12. Bitnami. *Sealed Secrets*. https://github.com/bitnami/sealed-secrets
13. Aqua Security. *Trivy documentation*. https://trivy.dev/
14. Gitleaks. *Gitleaks*. https://github.com/gitleaks/gitleaks
15. Semgrep. *Semgrep documentation*. https://semgrep.dev/docs/
16. Prometheus. *Documentation* (alerting rules, histogrammes). https://prometheus.io/docs/
17. Prometheus Operator. *kube-prometheus-stack*. https://github.com/prometheus-community/helm-charts
18. OWASP. *Top 10* et *Docker Security Cheat Sheet*. https://owasp.org/www-project-top-ten/, https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
19. HashiCorp. *Terraform documentation*. https://developer.hashicorp.com/terraform/docs
20. Ansible. *Ansible documentation* et *ansible-lint*. https://docs.ansible.com/

10 Annexe — Correspondance avec le dépôt

Élément	Emplacement
Mesures détaillées (méthode, limites)	`docs/mesures.md`
Journal d'apprentissage, incidents, questions d'entretien	`docs/apprentissage.md`
Politique de sécurité et exceptions	`docs/securite.md`
Décisions techniques datées	`CLAUDE.md` (section 10)
Pipeline CI / promotion	`.github/workflows/ci.yml`, `promote.yml`
Chart et état désiré	`k8s/helm/plateforme/`, `k8s/environments/`
Démonstration	`scripts/demo.sh`